

LC 1889: Min Space Wasted From Packaging

Amazon

2 Sigma

Problem Understanding

- n packages trying to place in m boxes
- 1 package $\rightarrow$ 1 box
- `packages[i]` $\rightarrow$ size of i^{th} package
- m suppliers who provides boxes of diff sizes
- `boxes[j]` $\rightarrow$ array of box sizes that j^{th} supplier gives
- Can only select 1 supplier such that space wasted is minimized
- **O/P** $\rightarrow$ min wasted total space or -1 if imp

Examples

Example 1

packages = [2, 3, 5]

If we choose second supplier	If we choose first supplier
boxes[1] = [2, 8] Available boxes matching size: [2, 8, 8]	boxes[0] = [4, 8] Available boxes matching size: [4, 4, 8]
Packages: [2, 3, 5]	Packages: [2, 3, 5]
Waste: (2-2) + (8-3) + (8-5) = 0 + 5 + 3 = 8	Waste: (4-2) + (4-3) + (8-5) = 2 + 1 + 3 = 6 (minimum waste)

Example 2

packages = [2, 3, 5]

boxes = [[1, 4], [2, 3], [3, 4]]

Note: When max size in boxes < max size of package → return -1

Method 1: Lower Bound Approach

1. Find max package
2. Sort packages
3. Sort each supplier boxes
4. If supplier largest box < max package (skip that supplier)
5. For every package find $\rightarrow$ `lower_bound`(boxes, package)
6. Add (box - package) to waste
7. Take minimum waste

Complexity: $O(n \log n + m \log m)$

Code: Lower Bound Method

TLE

```
class Solution {
public:
    int minWastedSpace(vector<int>& packages, vector<vector<int>>& boxes) {
        int n = packages.size();
        int m = boxes.size();
        const int MOD = 1e9+7;

        sort(packages.begin(), packages.end());
        long long ans = LLONG_MAX;

        for(auto &box : boxes) {
            sort(box.begin(), box.end());
            if(box.back() < packages.back()) continue;
            //ye supplier capable nhi he

            long long waste = 0;
            for(auto &p : packages) {
                int pos = lower_bound(box.begin(), box.end(), p) - box.begin();
                waste += box[pos] - p;
            }
            ans = min(ans, waste);
        }

        if(ans == LLONG_MAX) return -1;
        else return ans % MOD;
    }
};
```

Optimization: Two Pointers Approach

Without lower bound (will be more optimal).

Because packages and boxes are sorted, we can replace `lower_bound` → 2 pointer.

$O(m \log m) \rightarrow O(n + m)$

```
while (i < n && packages[i] <= boxes[j]) {  
    waste += boxes[j] - packages[i];  
    i++;  
}  
and then move to next box j++
```

Code: Two Pointers Method

TLE

```
class Solution {
public:
    int minWastedSpace(vector<int>& packages, vector<vector<int>>& boxes) {
        int n = packages.size();
        const int MOD = 1e9+7;

        sort(packages.begin(), packages.end());
        long long ans = LLONG_MAX;

        for(auto &box : boxes) {
            int m = box.size();
            sort(box.begin(), box.end());
            if(box.back() < packages.back()) continue;
            //ye supplier capable nhi he

            long long waste = 0;
            int i = 0;
            int j = 0;

            while(i < n && j < m) {
                if(packages[i] > box[j]) {
                    j++;
                }
                else {
                    waste += box[j] - packages[i];
                    i++;
                }
            }
            ans = min(ans, waste);
        }

        if(ans == LLONG_MAX) return -1;
        else return ans % MOD;
    }
};
```

Final Optimal Method: Prefix Sum + Binary Search

Logic Strategy:

OPTIMIZE KARNA PADEGA TLE DE RHA

ek kaam krte he. sabke liye lower bound nahi nikalte.

eg [2, 3, 4, 8] and box = [5, 10]

- then 2, 3, 4 ka lower bound to 5 hi he.
- to 2 ka lower bound 5 nikala.
- ab package me upper bound nikalenge 5 ka (i.e. the package jo uss box me nahi aa paayega).
- now $cnt = next - i$
- basically 2, 3, 4 sab 5 me jaayenge, to $waste += 5 * cnt = 5 * 3 = 15$
- $waste -= pref[next] - pref[i]$

Code: Prefix Sum & Binary Search

Correct

```

class Solution {
public:
    int minWastedSpace(vector<int>& packages, vector<vector<int>>& boxes) {
        int n = packages.size();
        const int MOD = 1e9+7;

        sort(packages.begin(), packages.end());
        long long ans = LLONG_MAX;

        vector<long long> pref(n + 1, 0);
        for(int i = 0; i < n; i++) {
            pref[i + 1] = pref[i] + packages[i];
        }

        for(auto &box : boxes) {
            sort(box.begin(), box.end());
            if(box.back() < packages.back()) continue;
            //ye supplier capable nhi he

            long long waste = 0;
            //OPTIMIZE KARNA PADEGA TLE DE RHA
            //ek kaam krte he
            //sabke liye lower bound nahi nikalte
            //eg [2,3,4,8] and box=[5,10]
            //then 2,3,4 ka lower bound to 5 hi he
            //to 2 ka lower bound 5 nikala
            //ab package me upper bound nikalenge 5 ka
            //i.e the package jo uss box me nahi aa paayega
            //now cnt=next-i
            //basically 2,3,4 sab 5 me jaayenge
            //to waste+=5*cnt=5*3=15
            //waste -= pref[next]-pref[i]

            int i = 0;
            while(i < n) {
                int j = lower_bound(box.begin(), box.end(), packages[i]) -
box.begin();
                int nexti = upper_bound(packages.begin() + i, packages.end(),
box[j]) - packages.begin() - 1;
                //end index of package that will go in lower bound box of start
index

                int cnt = nexti - i + 1;
                waste += (long long)box[j] * cnt;
                waste -= pref[nexti + 1] - pref[i];
                i = nexti + 1;
            }

            ans = min(ans, waste);
        }
    }
};

```

```
    }  
  
    if(ans == LLONG_MAX) return -1;  
    else return ans % MOD;  
}  
};
```